SOFTWARE METRICS & COMPLEXITY ANALYSIS

Smart Hospital Management System

Software Engineering Project Report

Name *– Abhay Sharma*
Roll no*. - 202401100200009*

1. Introduction

The Smart Hospital Management System (SHMS) is planned as one application for handling routine hospital tasks. These include patient registration, staff and patient login, appointment booking, consultation notes, prescriptions, vital signs, billing, notifications and administrative reports. The design also includes AI-assisted symptom triage, which can suggest a suitable department but must not replace a clinician's judgement.

Software metrics help describe how large a system is, how complicated its logic may be, and how well it is tested and maintained. For this project, they can help us estimate the work involved, decide which parts need more testing, and set targets that can be checked once development begins.

1.1 Objectives

- Estimate the size of the system using the functions described in the requirements.
- Identify workflows with more decision points and estimate their cyclomatic complexity.
- List the metrics that should be measured when the code and test cases are available.
- Suggest practical quality checks for a system that stores sensitive patient information.

1.2 Scope and assumptions

This report is based on the SHMS requirements and the UML diagrams prepared for the project. It assumes that patients and hospital staff sign in through role-based accounts, appointment slots cannot be booked twice, and changes to clinical records can be traced. It also assumes that services such as payments and notifications may occasionally fail. AI triage is treated as a support feature only. Since detailed data-element counts and file references are not available yet, the Function Point Analysis is an approximation rather than a formal IFPUG count.

2. Metrics Overview

Metric	What it tells us	How it helps this project
Function Points (FP)	Approximate amount of user-visible functionality, regardless of language.	Helps estimate the scope and effort early on.
Source Lines of Code (LOC)	Number of source-code lines after implementation.	Shows code size and growth; it should not be used alone to judge quality.
Cyclomatic Complexity (CC)	Estimated number of independent paths through a routine.	Helps choose which functions need more unit tests.
Halstead metrics	Measures based on the operators and operands in source code.	Can help compare the complexity of implemented methods.
Coupling and cohesion	How strongly modules depend on one another and how focused each module is.	Helps keep booking, records, billing and notifications easier to maintain.
Test coverage and defects	How much code tests exercise and what defects are found.	Helps track testing progress and reliability.
Quality characteristics	Areas such as functionality, reliability, security and maintainability.	Provides criteria for deciding whether a release is ready.

3. Preliminary Functional Size Estimation

Function Point Analysis estimates software size by looking at the data and transactions visible to users, rather than at the programming language. The counts below are based on the features planned for SHMS. For this first estimate, average weights have been used. A formal count would require more detail about each input, output, enquiry, and logical data file in the final requirements.

Function type	Assumed count	Average weight	Estimated points
Internal Logical Files (ILF)	9	10	90
External Interface Files (EIF)	3	7	21
External Inputs (EI)	12	4	48
External Outputs (EO)	8	5	40
External Inquiries (EQ)	10	4	40
Total Unadjusted Function Points (UFP)			239

The initial estimate is 239 unadjusted function points (UFP). This is a useful starting figure for discussion, but it has not been independently counted or checked against a completed system. The team should review the functions and their complexity against the final SRS before using this figure for cost or schedule planning.

3.1 Approximate LOC range

To get a rough idea of code size, this report assumes about 50 lines of source code per function point for a typical web application. Using that assumption gives the following result:

Calculation	Result
Estimated LOC = 239 FP × 50 LOC/FP	11,950 LOC
Rounded estimate	About 12 KLOC

Actual lines of code could be higher or lower depending on the language, framework, coding style, generated code and libraries reused. Once implementation begins, the estimate should be replaced with a count from the source repository or a code-analysis tool.

4. Cyclomatic Complexity Analysis

Cyclomatic complexity (CC) gives an estimate of how many independent paths exist through a piece of code. For a single connected routine, a common formula is CC = D + 1, where D is the number of decision points, such as if/else branches, switch cases or loops. The result helps the team decide how many different paths need attention during unit testing.

The values in the next table are based on likely decisions in each planned workflow. They are not readings from actual source code, so the final values may change after implementation.

Module / function	Assumed decisions (D)	Estimated CC (D + 1)	What should be tested
Login and access check	3	4	Valid and invalid login, account lock, unauthorised access and recovery.
Appointment booking	4	5	Invalid details, free or unavailable slots, confirmation and booking conflicts.
AI-assisted triage	3	4	Missing symptoms, ordinary suggestions, urgent flags and service failure.
Prescription creation	2	3	Valid details, missing fields and permission failures.
Payment processing	3	4	Successful payment, decline, timeout, duplicate callback and reconciliation.
Operational reports	2	3	Invalid dates or filters, empty results and successful export.

4.1 Worked example: appointment booking

For this example, assume the booking function checks four things: whether the user has permission, whether the appointment details are valid, whether the slot is still free, and whether the patient confirms the booking. With those four decision points, the estimate is:

Step	Calculation
Decision points	D = 4
Cyclomatic complexity	CC = D + 1 = 4 + 1 = 5
What it means	Plan tests for at least five independent paths in this simplified example.

A CC of 5 suggests that appointment booking has several paths that should be tested. One important case is when two patients try to book the same slot at nearly the same time. The system should check availability again when confirming the booking and save the reservation safely so both requests cannot succeed for the same slot.

5. Halstead Metrics

Halstead metrics use counts of operators and operands in the actual source code. Since the SHMS implementation is not available yet, accurate Halstead values cannot be calculated for this report. The main terms used in the calculations are listed below.

Symbol	Meaning
n_1	Number of different operators
n_2	Number of different operands
N_1	Total number of operator occurrences
N_2	Total number of operand occurrences
Vocabulary: $n = n_1 + n_2$	Total number of distinct operators and operands
Program length: $N = N_1 + N_2$	Total operator and operand occurrences
Volume: $V = N \times \log_2(n)$	Approximate information content of the routine
Difficulty: $D = (n_1 / 2) \times (N_2 / n_2)$	Estimate of how difficult the routine may be to understand or write
Effort: $E = D \times V$	Comparative effort indicator; not literal person-hours

After the code is written, the team can run a source-code analysis tool and record its name and version. Results are most useful when methods are compared using the same tool and counting rules; figures from different tools may not be directly comparable.

6. Object-Oriented Design and Dependency Analysis

Design measure	Observation from the current design	Recommendation
Number of named classes	The UML model contains 16 distinct classes.	Give each class one clear purpose and avoid unnecessary classes.
Inheritance	Patient, Doctor, Nurse, HospitalAdmin and Receptionist extend the abstract User class.	Keep shared account details in User and role-specific behaviour in the child classes.
Cohesion	Appointments, records, prescriptions, bills and services are separated by purpose.	Keep each class focused; move unrelated work into a separate service.
Coupling	Booking uses scheduling and notification services; clinical records connect to prescriptions.	Use clear interfaces and avoid links between unrelated modules.
Encapsulation	Patient and clinical data should not be changed directly from arbitrary screens.	Validate changes in the service/API layer and check permissions on the server.

These observations come from the current UML model. Exact object-oriented measures, such as coupling between objects, weighted methods per class, response for a class, and lack of cohesion, need the implemented code and a consistent measurement tool.

7. Quality Metrics and Suggested Targets

Quality area	Suggested project target	Why it matters
Unit test coverage	Aim for at least 80% statement coverage overall, with extra branch tests for booking, login and billing.	Coverage shows how much code has been exercised, but does not prove that it is defect-free.
Cyclomatic complexity	Prefer CC ≤ 10 per routine; review and simplify routines above 10 where practical.	Routines with many branches can take longer to test and maintain.

Quality area	Suggested project target	Why it matters
Appointment correctness	Concurrency tests should not allow duplicate active bookings for the same doctor and time slot.	Booking conflicts disrupt patients and hospital staff.
Access control	Test permissions for every protected API action and role.	Patient records must not be visible to unauthorised users.
Auditability	Record who changed an important clinical or administrative record, what changed and when.	This makes changes traceable if a mistake needs to be investigated.
Failure handling	Test payment, notification and AI-service timeouts; core data should remain consistent.	An external service failure should not corrupt the main system.
Defect density	Track confirmed defects per KLOC or function point once a stable baseline is available.	Trends over time are more useful than an unsupported one-time figure.

These are suggested targets for testing, not results the system has already achieved. The project team should confirm the targets with the supervisor and include them in the test plan.

8. Complexity and Risk Summary

Module	Main risk	Priority	Suggested control
Login and user roles	A permission error could expose private patient information.	High	Use central permission checks and test allowed and blocked access for every role.
Appointment scheduling	Simultaneous bookings or changes in availability may cause conflicts.	High	Check availability again during booking and use a transaction or locking strategy.
Medical records and prescriptions	Wrong or unauthorised changes could affect clinical decisions.	Very high	Validate input, limit access, keep an audit trail and test record integrity.
AI-assisted triage	A user may treat a suggestion as a diagnosis, or the service may be unavailable.	High	Explain its limitations, handle urgent symptoms carefully and leave clinical decisions to professionals.
Billing and payments	Timeouts or duplicate callbacks may leave an incorrect payment status.	High	Handle repeated callbacks safely and reconcile pending and failed payments.
Notifications and reports	Messages may be late or disclose more personal information than needed.	Medium	Retry delivery safely, keep message content minimal and record delivery status.

9. Conclusion

Based on the current requirements and diagrams, SHMS looks like a medium-sized application with several workflows that need careful handling. The early estimate is 239 unadjusted function points, or roughly 12 KLOC using the assumption stated above. The example workflows have estimated cyclomatic complexity values from 3 to 5. These figures are intended for planning and must not be treated as measurements from finished code.

During development, the main areas to focus on will be user permissions, the accuracy of patient records, avoiding duplicate appointment bookings, handling payments safely, and explaining the limits of AI-assisted triage. Once code and tests are available, the team should measure lines of code, cyclomatic complexity, Halstead metrics, test coverage and defect trends. Those real results will give a clearer picture of the system and help the team improve it over time.

10. References

1. ISO/IEC 25010:2023, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://www.iso.org/standard/78176.html

2. International Function Point Users Group (IFPUG), Function Point Analysis. https://ifpug.org/ifpug-standards/fpa

3. McCabe, T. J. (1982). Structured Testing: A Software Testing Methodology Using the Cyclomatic Complexity Metric. NIST Special Publication 500-99. https://www.nist.gov/publications/structured-testing-software-testing-methodology-using-cyclomatic-complexity-metric

4. McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, SE-2(4), 308–320.